CESAR SCHOOL

INFRAESTRUTURA DE SOFTWARE · 2026.2 · ADS

Desafio do PI: Container, TDD e Deploy automatizados

API Hemocentro
Relatório de entrega

Do doador ao paciente com segurança: triagem, doação, estoque por validade e transfusão com compatibilidade ABO/Rh. O sistema sobe com um comando, tem as regras protegidas por testes escritos antes do código e é publicado sozinho a cada merge na **main**.

Grupo do Projeto Integrador
Ewerton Guilherme da Silva · Pablo Arthur Eustáquio de Lima · João Ricardo Alves de Brito · Lucas Aprigio dos Santos · Saulo Eduardo Almeida dos Santos · Thiago Cardozo da Conceição · Eloi de Lima Sousa

github.com/egs3-coder/SO · hemocentro-api.onrender.com

Outubro de 2026

VISÃO GERAL

Resumo da entrega

Este relatório documenta, item por item, como o grupo atendeu ao **Desafio do PI: Container, TDD e Deploy automatizados**. Cada critério de avaliação vem acompanhado da decisão tomada, da justificativa técnica e da evidência real (prints das execuções, URL no ar e histórico de commits). O objetivo é que o professor consiga conferir tudo sem precisar perguntar nada ao grupo, que é exatamente a pergunta central do enunciado: *"se um colega clonar o repositório amanhã, ele consegue rodar, testar e publicar o sistema sem pedir ajuda a ninguém?"*

1 · Link do repositório

`https://github.com/egs3-coder/SO`

Repositório público, com o histórico completo de commits, pull requests e execuções do pipeline.

2 · URL pública

`https://hemocentro-api.onrender.com`

API no ar no Render. A raiz `/` mostra a versão publicada (SHA do commit).

3 · README

Na raiz do repositório, com as seções **Integrantes · Como rodar · Testes · Pipeline e deploy · Decisões · Uso de IA**, além de Evidências.

4 · Evidências

Pasta `docs/evidencias/` com 12 prints e o histórico red → green, incluindo a execução `vermelha` que barrou o merge e a `verde` que publicou.

01 **Integrantes** — quem fez e qual papel

02 **O sistema: API Hemocentro** — problema, arquitetura e regras

03 **Story map e tarefas** — backbone, Release 1, 3 histórias

04 **E1 · Container automatizado** — Dockerfile, compose, .env, como rodar

05 **E2 · TDD** — testes por nível, red → green, esteira

06 **E3 · Pipeline e deploy** — gates, GitHub Flow, Render, segredos, rollback

07 **Evidências do pipeline** — vermelha, verde, URL, rollback

08 **Decisões** — tabela da seção 6 do enunciado

09 **Uso de IA** — o que fez, como revisamos, prompts

10 **Problemas enfrentados e lições** — o que deu errado e como resolvemos

11 **Checklist dos critérios de avaliação** — onde está cada evidência

SEÇÃO 01

01 Integrantes

O grupo é o mesmo do Projeto Integrador. Os papéis abaixo são os do projeto. No desafio, o Product Owner conduziu a priorização das regras por risco e a configuração do pipeline, os desenvolvedores backend validaram as regras de negócio e os testes, e a dupla de frontend e UI cuidou da documentação, da organização das evidências e da revisão do README.

EG — **Ewerton Guilherme da Silva**
Product Owner

PA — **Pablo Arthur Eustáquio de Lima**
Backend Developer

JR — **João Ricardo Alves de Brito**
Backend Developer

LA — **Lucas Aprigio dos Santos**
Backend Developer

SE — **Saulo Eduardo Almeida dos Santos**
Backend Developer

TC — **Thiago Cardozo da Conceição**
Frontend / UI Designer

EL — **Eloi de Lima Sousa**
Frontend / UI Designer

Tecnologias utilizadas

Camada	Tecnologia e versão	Para que serve
Linguagem	Java 21 (LTS)	Versão de suporte longo, com *records* e *switch* modernos.
Framework	Spring Boot 3.5.16 (Web, Data JPA, Validation, Actuator)	API REST, persistência e endpoint de saúde.
Banco	PostgreSQL 17 + Flyway	Dados relacionais e schema versionado.
Testes	JUnit 5, AssertJ, Testcontainers, Cucumber 7	Unitário, integração com banco real e aceitação BDD.
Container	Docker (multi-stage) + Docker Compose	Sobe tudo do zero com um comando.
CI/CD	GitHub Actions + GitHub Container Registry	Esteira de build, testes, imagem e deploy.
Hospedagem	Render (Web Service por imagem + Postgres gerenciado)	Ambiente público.

02 O sistema: API Hemocentro

O tema do PI é um **hemocentro**. O ciclo do sangue tem pontos em que um erro de software vira risco direto à vida: coletar de um doador que não poderia doar, entregar uma bolsa vencida ou transfundir um tipo sanguíneo incompatível. Por isso o domínio é um ótimo caso para TDD. As regras são objetivas, verificáveis e cada uma tem uma consequência clara se falhar.

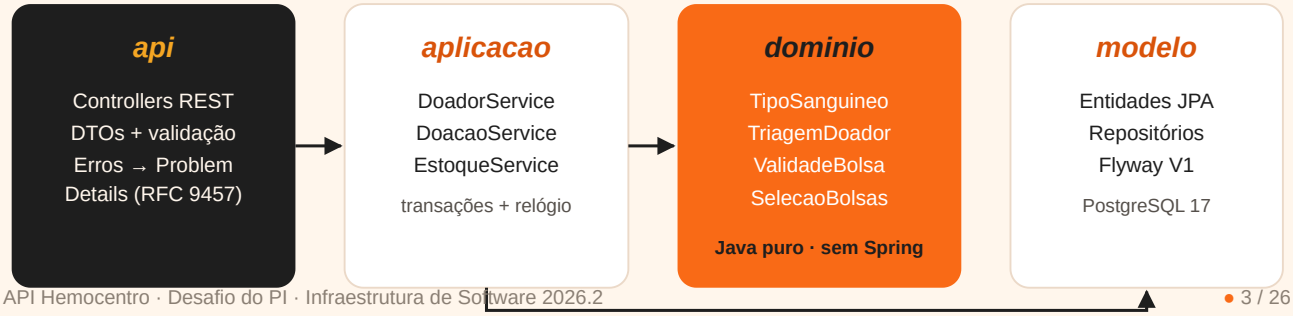
A API cobre o caminho mínimo do sangue, do doador ao paciente: **cadastrar o doador**, **triar** (verificar se ele pode doar), **registrar a doação** gerando uma bolsa com validade, **consultar o estoque** utilizável por tipo e **atender uma solicitação de transfusão** entregando apenas bolsas compatíveis e dentro da validade.

Endpoints

Método e rota	História	O que faz e o que devolve
`POST /doadores`	apoio	Cadastra o doador (nome, CPF, nascimento, sexo, peso, tipo sanguíneo). Devolve **201**; CPF repetido devolve **409**.
`GET /doadores/{id}/triagem`	H1	Avalia todas as regras de aptidão e devolve `{"apto": true|false, "motivos": [...]}`, com **todos** os motivos de uma vez.
`POST /doadores/{id}/doacoes`	H2	Se o doador estiver apto, registra a doação e gera a bolsa com validade de 35 dias (**201**). Se estiver inapto, devolve **422** com os motivos e nada é gravado.
`GET /estoque`	apoio	Quantidade de bolsas utilizáveis (disponíveis e dentro da validade) por tipo, com os 8 tipos sempre listados.
`POST /transfusoes`	H3	Recebe o tipo do receptor e a quantidade. Escolhe as bolsas pela regra de domínio e as marca como utilizadas (**201**). Sem estoque compatível suficiente, devolve **409** e não consome nada.
`GET /actuator/health`	infra	Saúde da aplicação (inclui a conexão com o banco). Usado pelo healthcheck do Compose e do Render.
`GET /`	infra	Mostra o serviço, a **versão no ar (SHA do commit)** e a lista de endpoints. É como o pipeline confere se o deploy e o rollback realmente aconteceram.

Arquitetura em camadas

O código foi separado para que as regras de negócio fiquem **isoladas do framework**. O pacote `dominio` é Java puro, sem Spring e sem banco. Isso permite testar cada regra em milissegundos, sem subir nada, que é o que torna o TDD viável no dia a dia.



Os *services* da camada `aplicacao` orquestram: buscam os dados nos repositórios, chamam a regra do domínio e gravam o resultado dentro de uma transação. O relógio (`Clock`) é injetado, então os testes controlam "que dia é hoje" e as regras de datas (intervalo, validade, idade) não dependem do dia em que o teste roda.

Regras de negócio implementadas

As regras de triagem se baseiam, de forma simplificada para o PI, na **Portaria de Consolidação MS nº 5/2017 (Anexo IV)**, que define os critérios técnicos para doação de sangue no Brasil.

Tarefa	Regra	Detalhe exato do que foi implementado
T1.1	Idade	Entre 16 e 69 anos. De 16 a 17 só com consentimento do responsável. Quem nunca doou só pode fazer a primeira doação até os 60 anos. O aniversário conta no dia exato (faz 16 hoje: pode; amanhã: ainda não).
T1.2	Peso	Mínimo de 50 kg (50,0 é apto; 49,9 não).
T1.3	Intervalo entre doações	Mínimo de 60 dias para homens e 90 dias para mulheres, contados da doação mais recente, mesmo que o histórico venha fora de ordem.
T1.4	Limite anual	No máximo 4 doações em 12 meses para homens e 3 para mulheres. Doações com mais de 12 meses não contam.
T2.1	Validade da bolsa	Concentrado de hemácias com anticoagulante CPDA-1 vale 35 dias após a coleta. A bolsa pode ser usada até o último dia, inclusive.
T3.1	Compatibilidade ABO/Rh	O doador só pode ter antígenos (A, B, D/Rh) que o receptor também tenha. Na prática, O- doa para todos, AB+ recebe de todos e O- só recebe de O-. Validado com as 26 combinações principais e as propriedades universais.
T3.2	Bolsa vencida	Nunca é entregue, nem contada como estoque utilizável.
T3.3	Prioridade de entrega	Primeiro o mesmo tipo do receptor (isogrupo), para preservar O- a quem só pode receber O-. Depois a bolsa que vence primeiro (FEFO: *first expired, first out*), para reduzir descarte.
T3.4	Estoque insuficiente	Se não houver bolsas compatíveis suficientes, a solicitação é recusada inteira. Entrega parcial esconderia a falta de sangue de quem pediu.
T3.5	Concorrência	As bolsas são travadas no banco (`SELECT … FOR UPDATE`) durante a transfusão, para que duas solicitações simultâneas não recebam a mesma bolsa.

03 Story map e tarefas

O story map organiza o sistema pela **jornada do sangue**. Na horizontal fica o *backbone*, as cinco grandes atividades na ordem em que acontecem. Na vertical, as histórias de cada atividade, das mais essenciais (em cima) às que podem esperar (embaixo). A **Release 1** é a primeira linha: o mínimo para o sangue ir do doador ao paciente com segurança. O arquivo completo está em `docs/story-map.md`.

1. Captar doador	2. Triar doador	3. Coletar	4. Armazenar	5. Distribuir
Cadastrar doador	**Verificar aptidão (idade, peso, intervalo)**	**Registrar doação e gerar bolsa**	**Controlar validade e estoque por tipo**	**Atender solicitação de transfusão**
Agendar doação	Registrar entrevista clínica	Registrar exames sorológicos	Alertar estoque crítico	Registrar reação transfusional
Campanhas e lembretes	Bloqueio temporário (viagem, tatuagem…)	Fracionar em hemocomponentes	Descartar bolsa vencida	Relatórios de consumo

Release 1 linha destacada · as linhas 2 e 3 ficam para releases futuras.

As três histórias da Release 1, quebradas em tarefas

H1 · Triagem do doador

Como **atendente do hemocentro**, quero **saber se um doador está apto** para **não coletar sangue de quem a norma proíbe**.

Tarefa	Regra	Teste que protege
T1.1	Idade 16–69; 16–17 com consentimento; 1ª doação até 60	`TriagemDoadorTest.Idade` (9 casos)
T1.2	Peso mínimo de 50 kg	`TriagemDoadorTest.Peso` (4 casos)
T1.3	Intervalo de 60 (H) / 90 (M) dias	`TriagemDoadorTest.Intervalo` (5 casos)
T1.4	Máximo de 4 (H) / 3 (M) doações em 12 meses	`TriagemDoadorTest.LimiteAnual` (4 casos)
T1.5	Endpoint `GET /doadores/{id}/triagem`	`triagem.feature` (3 cenários)

H2 · Registrar doação

Como **técnico de coleta**, quero **registrar a doação e gerar a bolsa** para **que ela entre no estoque com a validade correta**.

Tarefa	Regra	Teste que protege
T2.1	Bolsa vale 35 dias após a coleta	`ValidadeBolsaTest` (3 casos)

Tarefa	Regra	Teste que protege
T2.2	Doador inapto não pode doar (reaproveita H1)	`doacao.feature`
T2.3	Persistir doação + bolsa no banco	`BolsaRepositoryIT`
T2.4	Endpoint `POST /doadores/{id}/doacoes`	`doacao.feature` (2 cenários)

H3 · Atender solicitação de transfusão

Como **médico**, quero **receber bolsas compatíveis com o meu paciente** para **transfundir sem risco de reação hemolítica**.

Tarefa	Regra	Teste que protege
T3.1	Compatibilidade ABO/Rh de hemácias	`TipoSanguineoTest` (38 casos)
T3.2	Bolsa vencida nunca é entregue	`SelecaoBolsasTest`
T3.3	Isogrupo primeiro, depois FEFO	`SelecaoBolsasTest`
T3.4	Estoque insuficiente → recusa total	`SelecaoBolsasTest`
T3.5	Endpoint `POST /transfusoes` marca bolsas como utilizadas	`transfusao.feature` + `BolsaRepositoryIT`

04 E1 · Container automatizado

O objetivo do E1 é que o sistema suba do zero, com um comando, **igual em qualquer máquina**. Isso é a base do deploy: a mesma imagem que roda no computador do grupo é a que roda no Render, construída pelo mesmo Dockerfile.

Como rodar do zero

O único pré-requisito é ter o Docker instalado. Não é preciso instalar Java, Maven nem PostgreSQL na máquina.

```
# 1. baixar o projeto
git clone https://github.com/egs3-coder/SO.git && cd SO

# 2. criar a configuração local a partir do modelo (e trocar a senha)
cp .env.example .env

# 3. construir a imagem e subir API + banco
docker compose up -d --build

# 4. conferir: api e db devem aparecer como "healthy" (a API leva ~40 s)
docker compose ps
curl http://localhost:8080/actuator/health
```

1 · Dockerfile multi-stage

O Dockerfile tem **dois estágios**. O primeiro (`build`) usa uma imagem com JDK e Maven: baixa as dependências, compila e **roda os testes unitários** no `mvn package`. Se algum teste falhar, o build quebra e **nenhuma imagem é gerada**. O segundo estágio parte de uma imagem só com o JRE e copia apenas o `.jar` final.

```
FROM maven:3.9-eclipse-temurin-21-alpine AS build      # estágio 1: compila e testa
COPY pom.xml .
RUN  mvn -B -q dependency:go-offline                   # camada de dependências em cache
COPY src ./src
RUN  mvn -B package                                    # teste vermelho = sem imagem

FROM eclipse-temurin:21-jre-alpine                      # estágio 2: só o necessário
RUN  addgroup -S app && adduser -S app -G app
COPY --from=build /app/target/hemocentro.jar app.jar
USER app                                               # não roda como root
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

- **Cache de dependências:** o `pom.xml` é copiado antes do código. Enquanto ele não muda, a camada das dependências vem do cache e o build fica muito mais rápido.
- **Usuário sem privilégios:** a aplicação roda como o usuário `app`, não como root. Se alguém explorar uma falha na aplicação, não ganha controle total do container.
- **Versão embutida:** a esteira passa o SHA do commit como `APP_VERSION`. A rota `/` mostra esse valor, e é assim que o deploy e o rollback são conferidos.
- `.dockerignore`: exclui `target/`, `.git/`, `.env` e a documentação do contexto de build, o que deixa o build menor e evita copiar segredos para a imagem.

2 · compose.yaml: API + banco, volume, healthcheck e depends_on

```
services:
  db:
    image: postgres:17-alpine
    environment: { POSTGRES_PASSWORD: ${DB_PASSWORD:?Defina no .env} }
    volumes: [ "pgdata:/var/lib/postgresql/data" ]      # dados fora do container
    healthcheck: { test: pg_isready -U … -d …, interval: 5s, retries: 10 }
                                                        # sem "ports": banco não fica exposto
  api:
    build: .
    environment: { DB_HOST: db, DB_PASSWORD: ${DB_PASSWORD} } # "db" = nome do serviço
    ports: [ "8080:8080" ]
    depends_on: { db: { condition: service_healthy } }  # só sobe com o banco pronto
    healthcheck: { test: wget … /actuator/health | grep UP, start_period: 40s }
volumes:
  pgdata:
```

- **Volume nomeado `pgdata`:** os dados do Postgres ficam num volume gerenciado pelo Docker, separado do container. Um `docker compose down` remove os containers, mas os dados permanecem. Só o `down -v` apaga o volume.
- **Healthcheck nos dois serviços:** o banco responde ao `pg_isready`; a API, ao `/actuator/health`, que também verifica a conexão com o banco. O estado *healthy* significa "pronto para uso", não apenas "processo rodando".
- **`depends_on` com `service_healthy`:** a API só é iniciada depois que o banco está realmente aceitando conexões, o que evita falhas na primeira subida.
- **Rede do Compose:** o Compose cria uma rede própria para o projeto e registra cada serviço pelo nome. Por isso a API encontra o banco em `DB_HOST=db`, sem IP fixo. Só a API publica porta no host (8080); o banco fica acessível apenas dentro dessa rede.
- **Falha cedo sem senha:** `${DB_PASSWORD:?…}` faz o Compose parar com uma mensagem clara se o `.env` não existir, em vez de subir um banco sem senha.

3 · Configuração fora do código

Toda configuração vem de **variáveis de ambiente**. O `application.yml` só lê valores como `${DB_HOST:localhost}`, `${DB_PASSWORD:}` e `${PORT:8080}`. O repositório traz um `.env.example` com as variáveis necessárias e valores de exemplo, e o `.env` real está no `.gitignore`. Nenhuma senha está versionada. No Render, as mesmas variáveis são preenchidas pelo próprio provedor a partir do banco gerenciado (`fromDatabase` no `render.yaml`), e a porta vem de `PORT`, que o Render injeta.

```
# .env.example (versionado). Copie para .env (ignorado pelo Git)
DB_NAME=hemocentro
DB_USER=hemocentro
DB_PASSWORD=troque-esta-senha
API_PORT=8080
```

4 · Evidências do E1



Figura 1. `docker compose ps` com `hemocentro-api-1` e `hemocentro-db-1` em estado ***healthy***, seguido de `curl localhost:8080/estoque` respondendo com o JSON dos oito tipos sanguíneos. A API está de pé e conectada ao PostgreSQL.

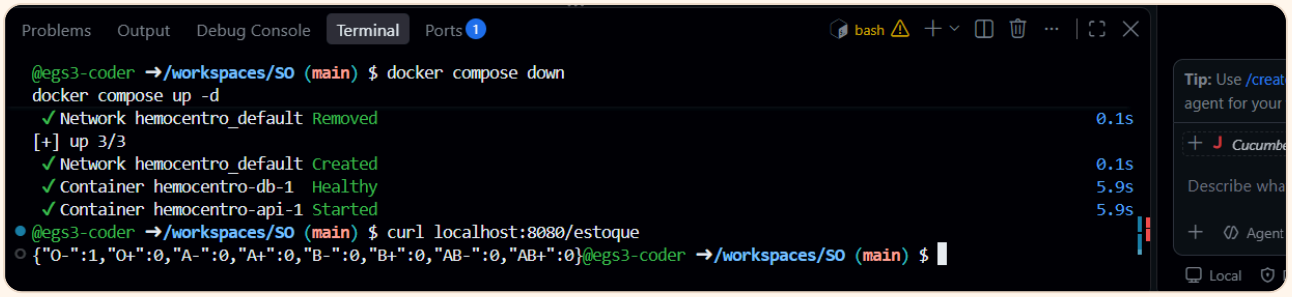


Figura 2. Prova de persistência do volume. Após registrar uma doação O-, os containers foram removidos com `docker compose down` (a rede aparece como *Removed*) e recriados com `docker compose up -d`. O estoque continua com `"O-":1`: o dado sobreviveu porque está no volume `pgdata`, não dentro do container.

05 E2 · TDD

Cada regra de negócio nasceu de um teste. O ciclo seguido foi o clássico do TDD (Beck, 2003): **red** (escrever um teste que falha, porque o código ainda não existe), **green** (escrever o mínimo de código para o teste passar) e **refactor** (melhorar o código mantendo os testes verdes). Isso ficou registrado no Git, com um commit para cada etapa.

Testes por nível

Unitário

39 métodos de teste que se desdobram em **72 casos** executados (testes parametrizados). Rodam em milissegundos, sem Spring nem banco. Pelo menos um por tarefa de regra.

Integração

2 testes em `BolsaRepositoryIT` contra um **PostgreSQL 17 real** criado pelo Testcontainers, com as migrations do Flyway aplicadas.

Aceitação (BDD)

8 cenários Gherkin em português, um arquivo `.feature` por história, executados pelo Cucumber via HTTP na aplicação inteira.

Unitários: o domínio isolado

Os testes unitários ficam em `src/test/java/.../dominio`. A classe `TriagemDoadorTest` usa classes `@Nested`, uma por tarefa (`Idade`, `Peso`, `Intervalo`, `LimiteAnual`), o que torna o relatório de testes um espelho do story map. Os testes exploram de propósito os **valores de fronteira**, onde os erros costumam aparecer: 49,9 kg e 50 kg; 59 e 60 dias; aniversário de 16 anos hoje e amanhã; bolsa que vence hoje e a que venceu ontem.

```
@ParameterizedTest(name = "{0} doa para {1}? {2}")
@CsvSource({ "O-, AB+, true", "O+, O-, false", "A-, A+, true", "AB+, O+, false", … })
void compatibilidadeDeHemacias(String doador, String receptor, boolean esperado) {
    assertThat(deSimbolo(doador).podeDoarPara(deSimbolo(receptor))).isEqualTo(esperado);
}
```

Integração: banco de verdade, não um banco "parecido"

Em vez de um banco em memória (H2), que tem SQL e comportamento diferentes do PostgreSQL, o teste de integração usa o **Testcontainers** para subir um `postgres:17-alpine` real, a mesma versão do `compose.yaml`. Ele confirma que as migrations do Flyway criam um schema compatível com as entidades JPA e que a consulta do estoque traz **só bolsas disponíveis e dentro da validade** (inclui a que vence hoje e exclui a vencida e a já utilizada).

Aceitação: cenários BDD em Gherkin

Os cenários descrevem o comportamento na linguagem do negócio (North, 2006). Cada passo chama a API por HTTP numa porta aleatória, com a aplicação inteira e o banco no ar, como um cliente faria.

```
# language: pt
Funcionalidade: H3 - Atender solicitação de transfusão
  Contexto:
    Dado o estoque possui as bolsas:
      | tipo | vence_em_dias |
      | O-   | 10            |
      | A+   | 5             |
      | A+   | 20            |
      | B+   | -1            |
  Cenário: Receptor A+ recebe a bolsa A+ que vence primeiro
    Quando o médico solicita 1 bolsa para um receptor "A+"
    Então a transfusão é atendida com 1 bolsa do tipo "A+"
    E a bolsa entregue vence em 5 dias
    E restam 2 bolsas utilizáveis no estoque
```

Nesse cenário, um único exemplo verifica quatro regras ao mesmo tempo: compatibilidade (A+ recebe A+), isogrupo (não gasta o O-), FEFO (entrega a de 5 dias, não a de 20) e exclusão da bolsa vencida (B+ com –1 dia não conta no estoque).

Histórico red → green

O histórico do repositório mostra, para cada tarefa, o commit `test: … - red` antes do `feat: … - green`. São **10 pares red → green**, na ordem de prioridade por risco. A lista completa está em `docs/evidencias/e2-historico-red-green.txt` e pode ser conferida com `git log --oneline --reverse`.

`2fecfcb`
chore: estrutura do projeto Spring Boot e story map da Release 1

`e90d03b`
test: compatibilidade ABO/Rh de hemácias (T3.1) - red

`2fc46a0`
feat: compatibilidade ABO/Rh por antígenos (T3.1) - green

`909d568`
test: validade de 35 dias da bolsa (T2.1) - red

`5cc6181`
feat: cálculo de validade e vencimento da bolsa (T2.1) - green

`9ecf8cf`
test: seleção de bolsas compatíveis, não vencidas e FEFO (T3.2, T3.3) - red

`7f80984`
feat: seleção por compatibilidade, validade, isogrupo e FEFO (T3.2, T3.3) - green

`5130a19`
test: recusa por estoque insuficiente (T3.4) - red

`666e8c5`
feat: recusa transfusão quando o estoque compatível não basta (T3.4) - green

`2661083 · 8615eac · 18c0d3c · 88cda0a`
test: idade (T1.1), peso (T1.2), intervalo (T1.3) e limite anual (T1.4) - red, cada um seguido do seu feat - green

`ae4fc0e`
test: integração do estoque e doações com PostgreSQL via Testcontainers (T2.3, T3.5) - red

`ee8b424`
feat: entidades JPA, repositórios e migration Flyway (T2.3, T3.5) - green

`7227e37`
test: cenários de aceitação BDD (Gherkin) para H1, H2 e H3 - red

`19d1227`
feat: API REST de doadores, triagem, doações, estoque e transfusões (H1, H2, H3) - green

Quais regras ganharam teste primeiro e por quê (risco)

A ordem dos testes não foi aleatória: seguiu o **tamanho do dano** que um erro causaria. Primeiro o que pode matar um paciente, depois o que desperdiça sangue, por fim o que protege o doador.

#	Regra	Por que veio nessa posição
1º	Compatibilidade ABO/Rh (T3.1)	Transfundir hemácias incompatíveis causa reação hemolítica aguda, que pode ser fatal. É o maior risco do sistema e não tem como "desfazer" depois.
2º	Validade e bolsa vencida (T2.1, T3.2)	Sangue vencido perde qualidade e pode estar contaminado. Também é risco direto ao paciente.
3º	Isogrupo, FEFO e estoque insuficiente (T3.3, T3.4)	Gastar O- à toa deixa sem sangue quem só pode receber O-. Entrega parcial esconde a falta de sangue do médico. Não matam diretamente, mas causam desabastecimento.
4º	Triagem do doador (T1.1 a T1.4)	Protege a saúde do doador (anemia, desmaio) e o cumprimento da norma. É importante, mas o dano é menor e o processo real tem a entrevista clínica como segunda barreira.

O que ficou sem teste (e por quê)

- **Funcionalidades fora da Release 1:** entrevista clínica, exames sorológicos, bloqueios temporários, fracionamento, alertas de estoque e relatórios (linhas 2 e 3 do story map). Não foram implementadas, portanto não têm teste.
- **Validação real de CPF:** só o formato (11 dígitos) é verificado, não os dígitos verificadores.
- **Concorrência sob carga:** existe a trava pessimista no banco, mas não há teste disparando transfusões simultâneas. Seria o próximo teste de integração a escrever.
- **Controllers e DTOs:** não têm teste unitário próprio, porque são cobertos de ponta a ponta pelos cenários de aceitação via HTTP.

Testes na esteira: o gate do merge

O job `testes` do GitHub Actions roda `mvn verify`, que executa os três níveis (unitário com Surefire; integração e aceitação com Failsafe), **em todo push e em todo pull request**. Esse job é um **check obrigatório** na proteção da `main`: se ficar vermelho, o botão de merge fica bloqueado. Os relatórios (Surefire, Failsafe e o HTML do Cucumber) ficam anexados a cada execução como artefato.

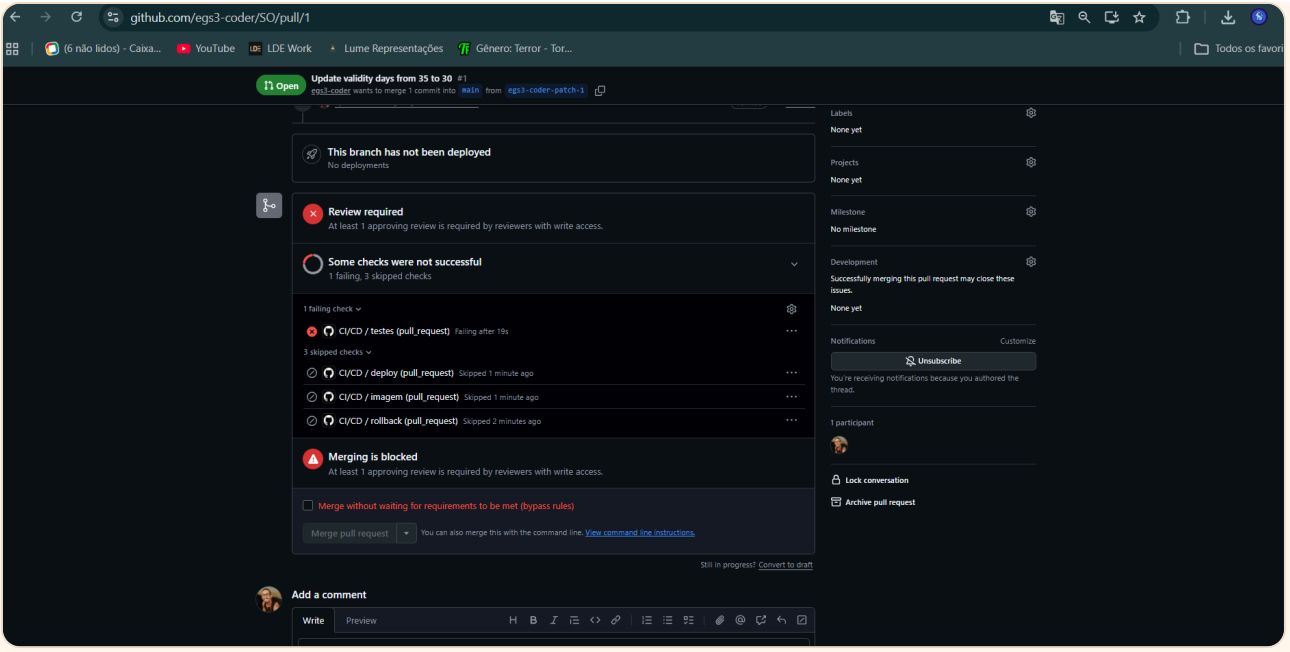
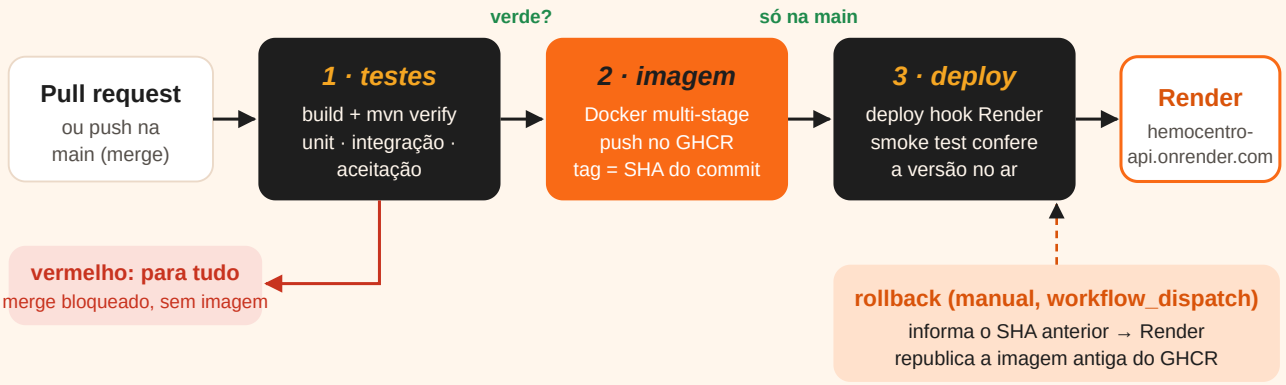


Figura 3. Teste barrando uma mudança errada. No PR #1 a validade da bolsa foi alterada de propósito de 35 para 30 dias. O check `CI/CD / testes` falhou (o `ValidadeBolsaTest` espera 35 dias), os jobs `imagem` e `deploy` foram pulados e o GitHub exibe ***"Merging is blocked"*** . O PR foi fechado sem merge.

06 E3 · Pipeline e deploy

A regra do E3 é que **cada merge na `main` chegue sozinho a um ambiente público, e só com a esteira verde**. Ninguém do grupo publica nada à mão: o caminho para produção é sempre o pull request, os testes, a imagem e o deploy automático. É o *deployment pipeline* descrito por Humble e Farley (2010).

O pipeline: build → testes → imagem → deploy



Job	Quando roda	O que faz
testes	Todo PR e todo push na `main`	Checkout, Java 21 (Temurin) com cache do Maven e `mvn -B verify`. Publica os relatórios como artefato. É o **gate**: os outros jobs têm `needs: testes`.
imagem	Só em push na `main` (merge), se `testes` passou	Login no GHCR com o `GITHUB_TOKEN` da própria execução, build do Dockerfile multi-stage com cache e push com duas tags: `:SHA` (imutável, usada no deploy e no rollback) e `:latest`.
deploy	Depois de `imagem`	Executa `scripts/deploy-render.sh`: chama o deploy hook do Render com a URL da imagem `:SHA` e faz um *smoke test*, consultando a URL pública até ela responder com `"versao":"<SHA>"`. Se a versão nova não aparecer em 10 minutos, o job falha.
rollback	Manual (*Run workflow*)	Recebe o SHA de uma versão anterior e republica aquela imagem, com o mesmo script e a mesma verificação.

Dois detalhes deixam a esteira mais segura. O `concurrency` impede dois deploys ao mesmo tempo e cancela execuções antigas do mesmo PR. O `permissions` restringe o token ao mínimo (`contents: read`, `packages: write`).

Fluxo de branch: GitHub Flow aplicado de verdade

Adotamos o **GitHub Flow**: a `main` é sempre publicável; toda mudança nasce numa branch curta, vira um pull request, passa pelos testes e só então entra na `main`, que dispara o deploy. Isso foi aplicado na prática, como mostram os PRs do repositório:

- **PR #1** (`egs3-coder-patch-1`): mudança errada de propósito (validade 30 dias). Testes vermelhos, merge bloqueado, PR fechado.
- **PR #2** (`egs3-coder-patch-2`): atualização do README. Testes verdes, merge feito, deploy automático da versão `4566e43`.

A `main` está **protegida** por uma regra de branch: exige pull request antes do merge e exige que o check `testes` esteja verde (e atualizado com a `main`). Não é possível dar push direto nem fazer merge com teste vermelho.

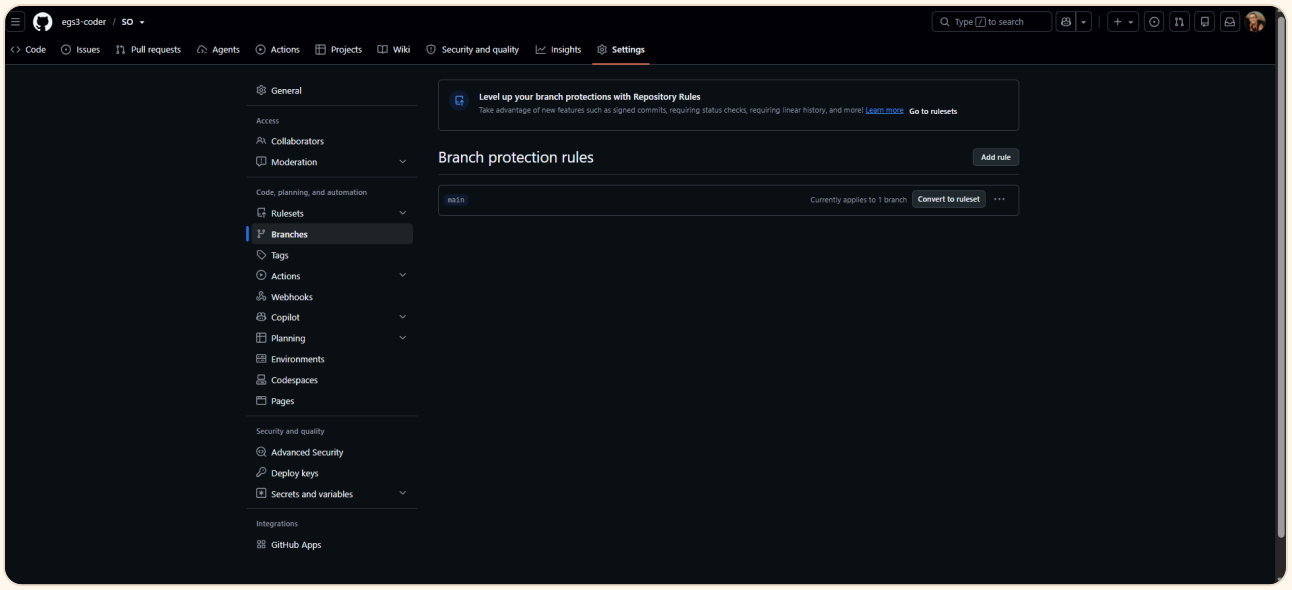


Figura 4. Regra de proteção da branch `main` em *Settings* → *Branches* do repositório.

Deploy automático no Render

O ambiente foi criado a partir do `render.yaml` (Blueprint), que descreve como código dois recursos: o **Web Service** `hemocentro-api`, que roda a imagem `ghcr.io/egs3-coder/so` com healthcheck em `/actuator/health`, e o **PostgreSQL gerenciado** `hemocentro-db`. As variáveis de conexão (`DB_HOST`, `DB_USER`, `DB_PASSWORD` ...) são ligadas automaticamente ao banco pelo próprio Render. O `autoDeploy` está **desligado de propósito**: quem decide quando publicar é a esteira, e ela só publica depois dos testes.

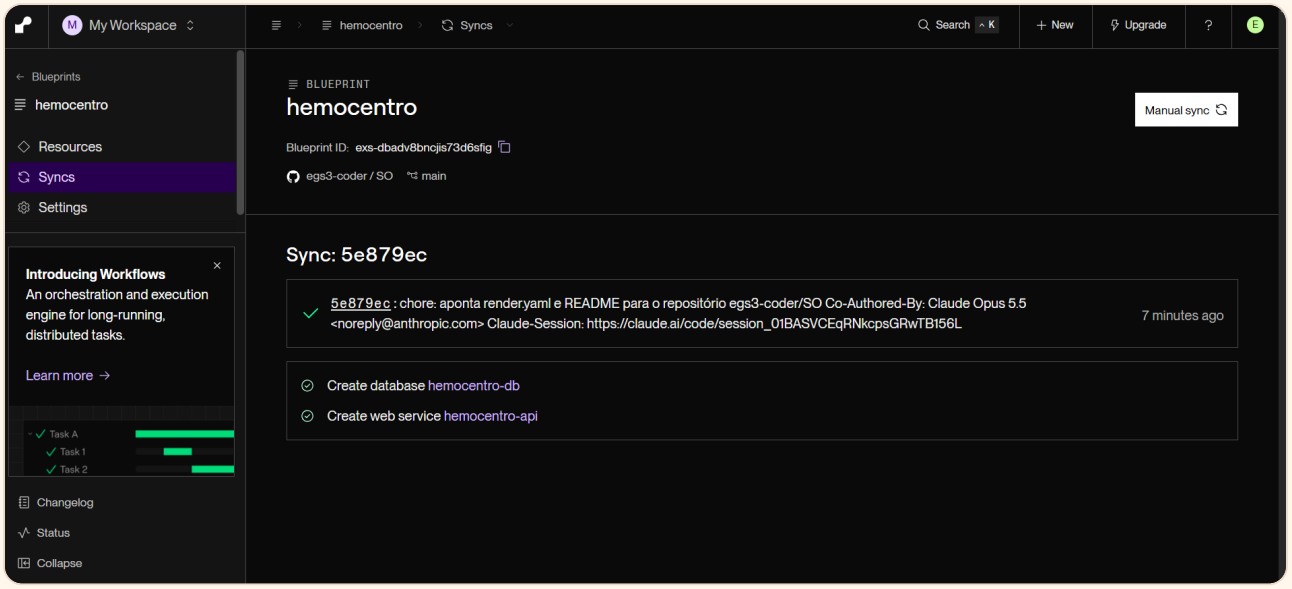


Figura 5. Blueprint `hemocentro` sincronizado no Render a partir do repositório `egs3-coder/SO`: criou o banco `hemocentro-db` e o web service `hemocentro-api`.

Onde ficam os segredos

Segredo ou configuração	Onde fica	Por quê
URL do deploy hook do Render	**GitHub Secrets** `RENDER_DEPLOY_HOOK_URL`	Quem tem essa URL consegue publicar. Fica criptografada no cofre do GitHub e aparece mascarada (`***`) nos logs.

Segredo ou configuração	Onde fica	Por quê
Senha do banco em produção	**Render** (gerada pelo provedor)	Nunca passa pelo repositório nem pelo GitHub. O Render injeta como variável de ambiente no container.
Senha do banco local	`.env` (no `.gitignore`)	Cada desenvolvedor tem a sua. O repositório só tem o modelo `.env.example`.
Token do GHCR	`GITHUB_TOKEN` automático	Gerado pelo Actions em cada execução e expira ao final. Não há token pessoal guardado.
URL pública	GitHub **Variables** `APP_URL`	Não é segredo, mas é configuração do ambiente, não do código.

Como voltar à versão anterior (rollback)

Cada versão publicada fica guardada no GitHub Container Registry com a **tag do SHA do commit**. Voltar para uma versão anterior, portanto, não exige recompilar nada: basta mandar o Render rodar a imagem antiga.

1. No repositório, abrir **Actions → CI/CD → Run workflow**.
2. Informar o SHA completo (40 caracteres) da versão desejada, por exemplo `5e879eced2cc3e508fd9d8314a108d24acd35f74`.
3. O job `rollback` chama o deploy hook com a imagem `ghcr.io/egs3-coder/so:<SHA>` e só fica verde quando a URL pública responde com aquela versão (cerca de 3 minutos).
4. Plano B: no painel do Render, em *Events*, é possível clicar em *Rollback* num deploy anterior.

O rollback foi testado nos dois sentidos: da versão atual `4566e43` para a anterior `5e879ec` (execução #6) e de volta para a `4566e43` (execução #7). As evidências estão na próxima seção.

07 Evidências do pipeline

As execuções abaixo são reais e podem ser conferidas na aba **Actions** do repositório. A ordem conta a história completa: primeiro deploy, mudança errada barrada, mudança correta publicada e rollback.

Exec.	Evento	Resultado	O que prova
#1	push na main: `5e879ec`	verde	Primeiro deploy automático. Testes, imagem e deploy; versão `5e879ec` no ar.
#2	PR #1: validade 35 → 30 dias	vermelho	**Execução vermelha que barrou o deploy.** Testes falharam após 19 s; imagem e deploy pulados; merge bloqueado.
#3	PR #2: atualização do README	verde	Testes do PR verdes (50 s), liberando o merge.
#4	merge do PR #2: `4566e43`	verde	**Execução verde que publicou.** testes 47 s → imagem 20 s → deploy 2 min 45 s; total 4 min 1 s.
#6	rollback para `5e879ec`	verde	Versão anterior de volta ao ar em 2 min 41 s.
#7	rollback para `4566e43`	verde	Retorno à versão atual em 2 min 43 s.

A execução #5 foi uma tentativa de rollback com o SHA colado incompleto: o Render recusou a imagem inexistente e o job falhou em 6 s, sem afetar a versão no ar. Detalhes na seção 10.

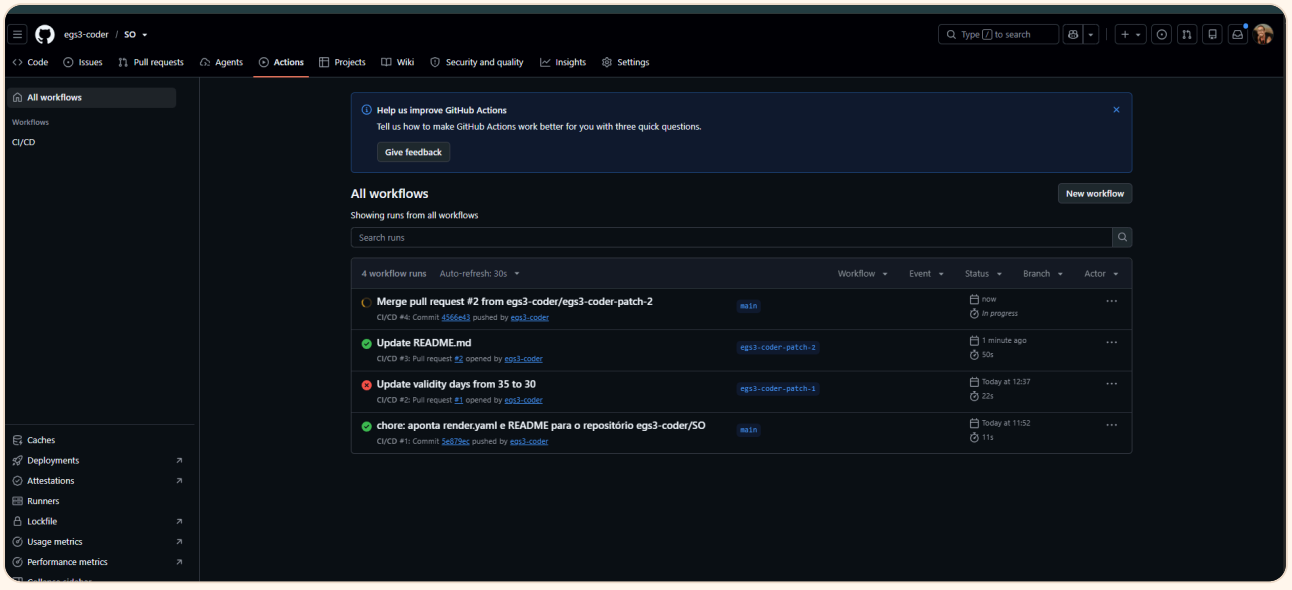


Figura 6. Lista de execuções do workflow CI/CD: a verde do primeiro deploy (#1), a ***vermelha*** do PR #1 (#2), a verde dos testes do PR #2 (#3) e o merge publicando a nova versão (#4, ainda em andamento no momento do print).

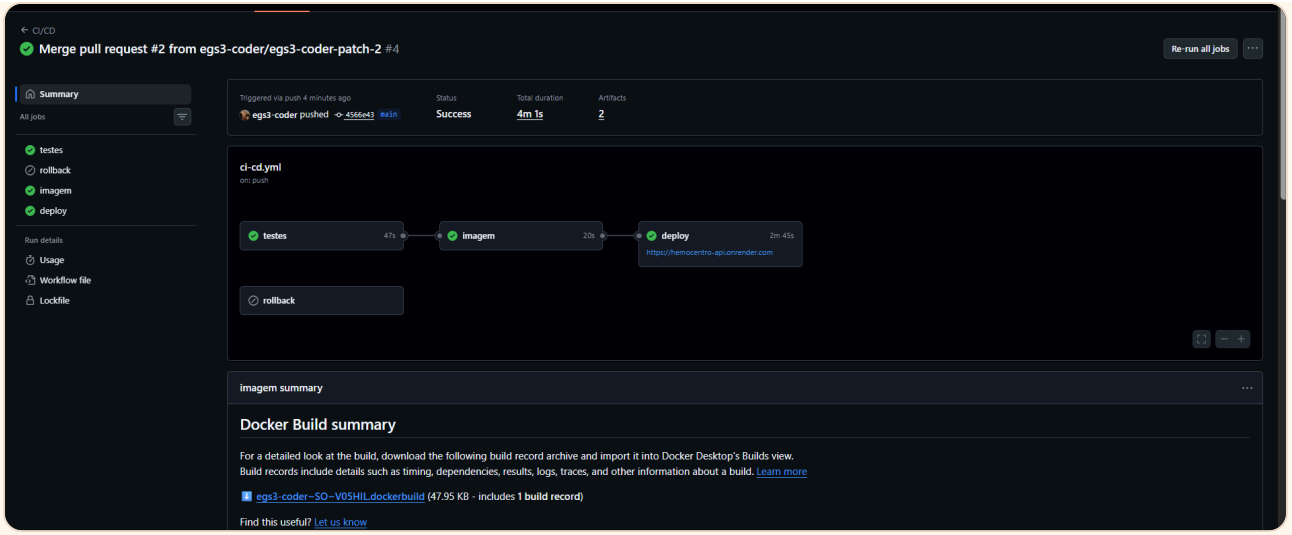


Figura 7. Execução **verde que publicou** (#4, merge do PR #2). Status *Success*, duração total de 4 min 1 s, com `testes` → `imagem` → `deploy` encadeados. O job `deploy` exibe a URL do ambiente publicado; `rollback` aparece como pulado porque só roda sob demanda.

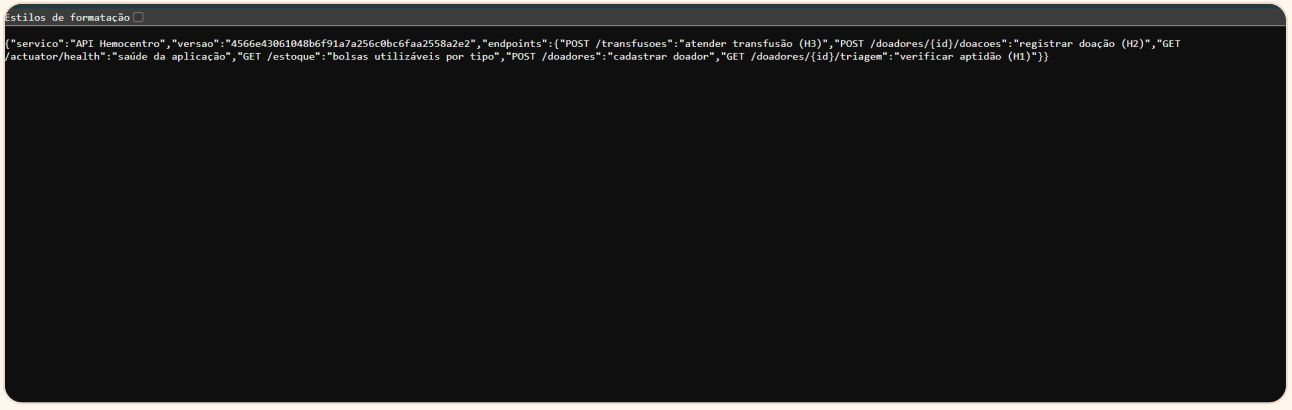


Figura 8. URL pública no ar: `https://hemocentro-api.onrender.com` respondendo com `"versao":"4566e43061048b6f91a7a256c0bc6faa2558a2e2"`, o SHA exato do merge do PR #2. Isso prova que a versão no ar é a que a esteira publicou.

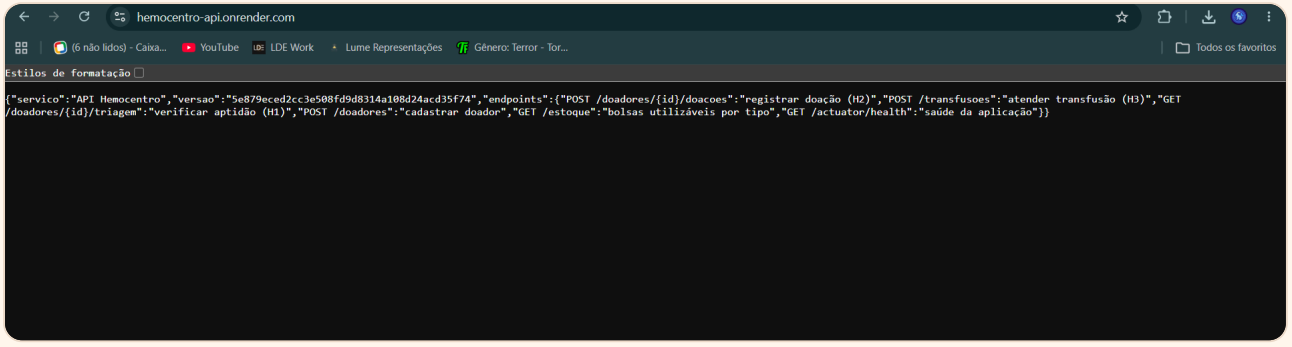


Figura 9. A mesma URL servindo a versão do primeiro deploy, `5e879eced2cc…`. É a versão para a qual o rollback voltou.

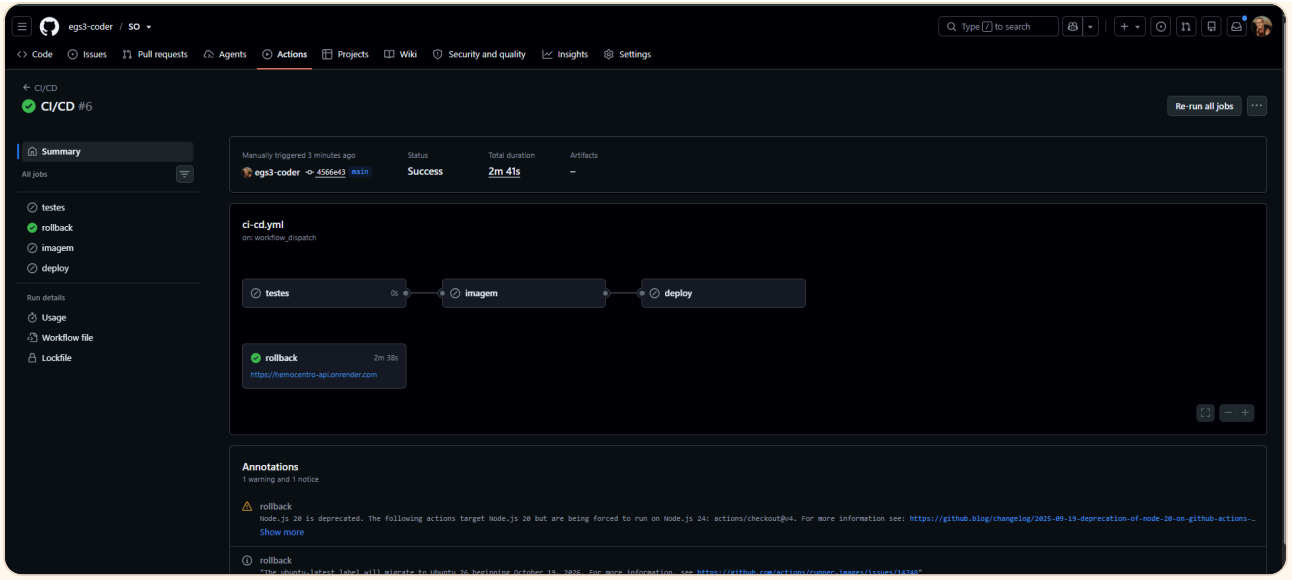


Figura 10. Rollback (execução #6, disparo manual via *workflow_dispatch*): o job `rollback` ficou verde em 2 min 38 s, depois de confirmar que a URL pública passou a responder com a versão anterior. Os jobs normais aparecem como pulados.

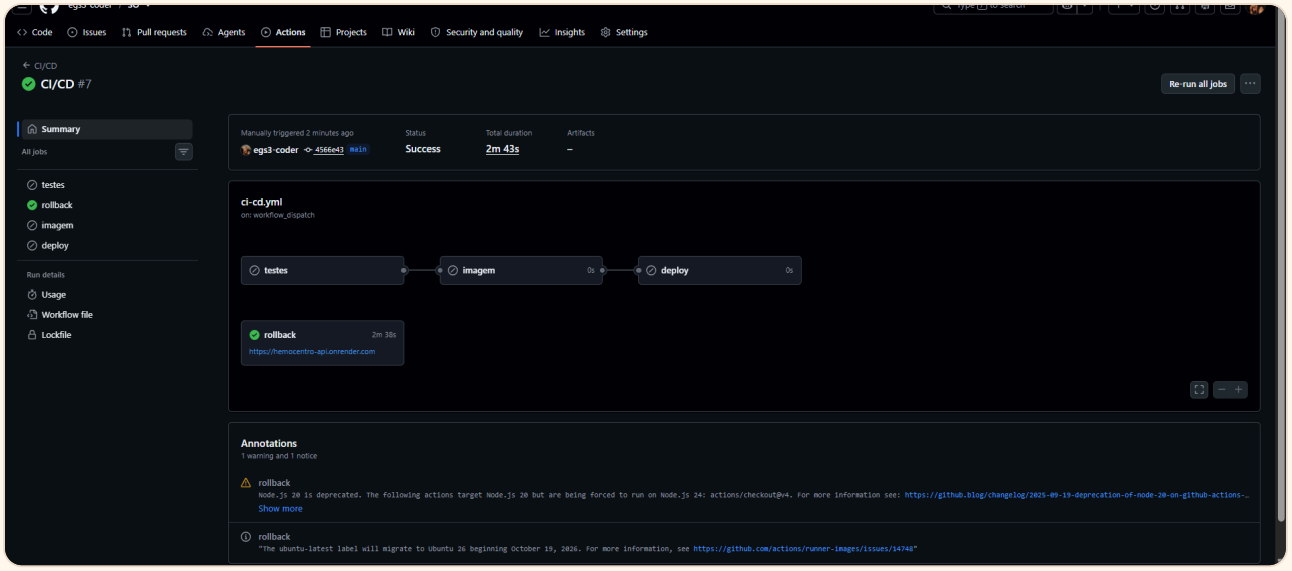


Figura 11. Volta para a versão atual (execução #7, 2 min 43 s) e a URL respondendo novamente com `4566e43…`. O rollback funciona nos dois sentidos.

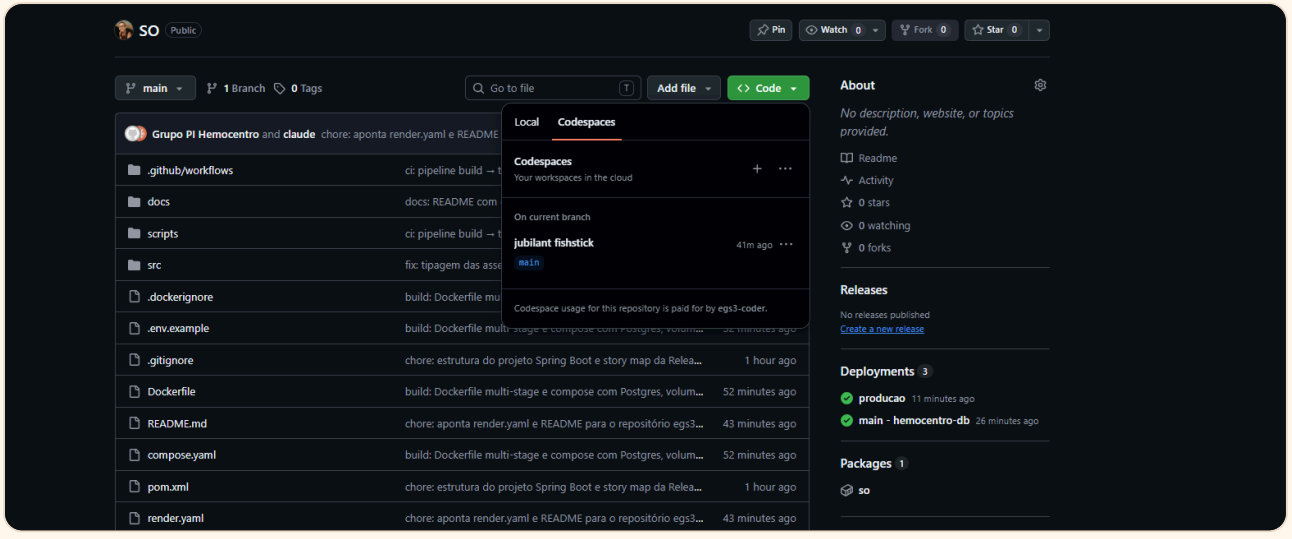


Figura 12. Página do repositório com o ambiente `producao` registrado em *Deployments* (marcado como bem-sucedido) e o pacote `so` publicado em *Packages* (GHCR).

08 Decisões

Seguindo a seção 6 do enunciado, cada linha registra **o que escolhemos, as alternativas consideradas e o porquê**. A mesma tabela está no README do repositório.

Decisão	Opções consideradas	O que escolhemos e por quê
Linguagem e framework	Java/Spring, Node/Nest, Python/FastAPI	**Java 21 + Spring Boot 3.5**: é a stack trabalhada em aula (ola-java) e tem o ecossistema de testes mais maduro para o que o desafio pede: JUnit 5, Testcontainers e Cucumber integrados ao Spring.
Imagem base	`eclipse-temurin:21-jre`, `-jre-alpine`, `openjdk`	`eclipse-temurin:21-jre-alpine` no runtime: só o JRE, sem compilador nem Maven, sobre o Alpine, que é mínimo. Imagem menor, mais rápida de baixar no deploy e com menos pacotes onde vulnerabilidades possam existir. A imagem oficial `openjdk` foi descontinuada. Roda como usuário não-root.
Multi-stage	Imagem única com JDK; build fora do Docker; multi-stage	**Multi-stage**: o 1º estágio compila e roda os testes unitários (teste vermelho = sem imagem); o 2º leva só o `.jar`. O que vai para produção é exatamente o que foi testado, e as ferramentas de build não chegam a produção.
Banco de dados	PostgreSQL, MySQL, MongoDB	**PostgreSQL 17**: os dados são relacionais (doador → doação → bolsa), precisamos de transações e de `SELECT … FOR UPDATE` para a concorrência das transfusões. O Render oferece Postgres gerenciado gratuito.
Onde ficam os dados	Volume nomeado, bind mount	**Volume nomeado** `pgdata`: os dados sobrevivem ao `docker compose down` (testado, figura 2) e não dependem da estrutura de pastas nem das permissões da máquina de cada integrante, como aconteceria com um bind mount.
Schema	`ddl-auto: update`, Flyway, Liquibase	**Flyway** com `V1__schema_inicial.sql`, e o Hibernate apenas valida. O schema fica versionado no Git e é idêntico no PC, no teste e em produção. O `update` automático pode alterar o banco de forma imprevisível.
Rede do Compose	Rede padrão, rede customizada, portas no host	**Rede padrão do Compose**: a API encontra o banco pelo nome do serviço (`DB_HOST=db`) via DNS interno. O banco não publica porta no host, só a API (8080), o que reduz a superfície exposta.
Ordem de subida	`depends_on` simples, script *wait-for*, healthcheck	`depends_on: service_healthy` + healthchecks (`pg_isready` e `/actuator/health`). O `depends_on` simples só espera o container iniciar, não o banco ficar pronto.
Configuração	Valores no código, `.env`, variáveis de ambiente	**Variáveis de ambiente** lidas pelo `application.yml`, com `.env.example` versionado e `.env` no `.gitignore`. A mesma imagem roda em qualquer ambiente mudando só as variáveis (12-factor).
Testes: ordem	Qual regra testar primeiro	**Compatibilidade ABO/Rh primeiro**: um erro ali põe um paciente em risco de morte. Depois validade, seleção de bolsas e por fim a triagem, conforme a tabela de risco da seção 5.
Testes: níveis	Só unitário, H2 em memória, banco real	**Unitário** no domínio puro (rápido); **integração com Postgres real** via Testcontainers, porque o H2 não se comporta igual ao Postgres; **aceitação em Gherkin** com Cucumber, legível por quem conhece o negócio.

Decisão	Opções consideradas	O que escolhemos e por quê
Fluxo de branch	Git Flow, GitHub Flow, GitLab Flow	**GitHub Flow**: grupo pequeno, uma única versão no ar e deploy a cada merge. O Git Flow (develop, release, hotfix) é pensado para várias versões mantidas em paralelo, e aqui seria burocracia sem ganho. O GitLab Flow acrescenta branches por ambiente, e temos um só.
Ferramenta de CI/CD	GitHub Actions, GitLab CI, Jenkins	**GitHub Actions**: o código já está no GitHub, não há servidor para manter (ao contrário do Jenkins), é gratuito para repositório público e o GHCR vem integrado com o mesmo token.
Provedor de deploy	Render, Railway, Fly.io	**Render**: plano gratuito com deploy por imagem Docker, Postgres gerenciado no mesmo lugar, deploy hook para a esteira controlar a publicação e Blueprint (`render.yaml`) que cria tudo a partir do repositório.
Disparo do deploy	Auto-deploy do provedor, deploy hook pela esteira	**Deploy hook chamado pela esteira**, com `autoDeploy: false` : o Render só publica o que passou pelos testes. O job ainda confere a versão no ar antes de ficar verde.
Segredos	`.env` local, secrets da ferramenta	**Deploy hook no GitHub Secrets**; senha do banco de produção gerada e guardada pelo Render; `.env` local no `.gitignore` ; GHCR com token temporário. Nenhum segredo no repositório.
Rollback	Painel do Render, reverter commit, republicar imagem antiga	**Republicar a imagem antiga pelo SHA** via workflow manual: cada versão já está no GHCR, então voltar não exige rebuild nem novo commit e leva cerca de 3 minutos. Reverter o commit também funcionaria, mas é mais lento (passa por toda a esteira). O painel do Render fica como plano B.

09 Uso de IA

O enunciado permite o uso de IA, desde que declarado: *"a IA rascunha; vocês validam e assinam"*. O grupo usou o **Claude (Anthropic)** como assistente durante todo o desafio. Abaixo está, com transparência, o que a IA produziu, o que o grupo fez e como a revisão aconteceu.

O que a IA fez

- Propôs o story map e as regras de negócio do hemocentro a partir do tema dado pelo grupo, com base na norma do Ministério da Saúde.
- Escreveu os testes e o código no ciclo red → green, commit a commit.
- Criou o Dockerfile, o `compose.yaml`, o workflow do Actions, o `render.yaml` e os scripts de deploy e de proteção da branch.
- Guiou a configuração do GitHub, do Render e do Codespace passo a passo, a partir dos prints que enviamos.
- Diagnosticou os problemas que surgiram (seção 10) e redigiu o README e este relatório.

O que o grupo fez e validou

- Definiu o tema, a stack (Java + Spring Boot) e o provedor (Render).
- Criou o repositório, configurou secrets, variáveis, Blueprint e proteção da `main`.
- Executou tudo de verdade: esteira com os três níveis de teste, deploy, API testada no ar e localmente.
- Provocou uma falha de propósito (35 → 30 dias) para confirmar que o teste barra o merge.
- Testou o rollback nos dois sentidos e a persistência do volume.
- Conferiu as regras (idade, peso, intervalos, ABO/Rh) contra a norma.

Como a revisão aconteceu

A principal forma de revisão foi **executar e observar o resultado**, não apenas ler o código. Um exemplo: o ambiente da IA não tinha acesso ao Maven Central nem ao Docker Hub, então ela não conseguia compilar a parte Spring. Ela declarou isso e validou as regras de domínio compilando-as em Java puro (88 verificações equivalentes aos testes unitários). A primeira compilação completa, com Spring, banco e Cucumber, aconteceu no GitHub Actions, sob observação do grupo, e passou. Cada passo de configuração foi confirmado por print antes de seguir para o próximo, e os erros encontrados foram investigados pelo log, não por tentativa e erro.

Prompts principais

#	Prompt (resumido)	Resultado
1	*"Faça esse desafio (PDF do enunciado) atendendo todos os critérios; o foco primeiro é o desafio e seus componentes principais para teste."* Respostas às perguntas da IA: Java + Spring Boot, projeto do zero, GitHub Actions + Render.	Escopo e stack definidos.
2	*"O tema seria em relação a hemocentro."*	Story map, regras, testes e código com histórico red → green.
3	*"O nome do repositório é egs3-coder/SO."*	`render.yaml` e README apontando para o repositório.
4	*"Me diga o passo a passo para eu rodar ele na versão online pelo GitHub."*	Roteiro: Codespace, Actions, GHCR, Render, secrets, proteção da main.

#	Prompt (resumido)	Resultado
5	Envio de prints de cada etapa (Actions, Render, PRs, Codespace) com *"o que eu faço agora?"*	Próximo passo e diagnóstico de cada erro.
6	*"Baseado nos prints que eu mandei, faça os arquivos de evidência."* e *"faça um PDF com a identidade visual da CESAR, detalhado e justificado."*	Pasta `docs/evidencias`, README final e este relatório.

> ***Responsabilidade***
>
> Todo o conteúdo entregue foi executado, conferido e aprovado pelo grupo, que assume a autoria e a responsabilidade pelas decisões técnicas aqui justificadas.

10 Problemas enfrentados e lições

Nem tudo funcionou de primeira, e os problemas também mostram que o processo foi feito de verdade. Cada um foi diagnosticado pelo log, corrigido e documentado.

1 · Primeiro deploy falhou

Sintoma: no primeiro push, `testes` e `imagem` passaram, mas o job `deploy` falhou. **Causa:** o secret `RENDER_DEPLOY_HOOK_URL` e a variável `APP_URL` ainda não existiam, porque o ambiente no Render só pode ser criado depois que a imagem existe no GHCR. **Solução:** criar o Blueprint, cadastrar o secret e a variável e usar *Re-run failed jobs*. **Lição:** o script de deploy valida os dois valores no início e falha com mensagem clara, em vez de publicar "no escuro".

2 · Rollback com SHA incompleto (execução #5)

Sintoma: o job `rollback` falhou em 6 s com `exit code 22`. **Causa:** o código de saída 22 é o `curl` recebendo um erro 4xx: o SHA foi colado incompleto, e o Render recusou a imagem com uma tag inexistente no GHCR. **Solução:** rodar novamente com o SHA completo de 40 caracteres (execução #6, verde). **Lição:** a falha foi segura. Como o Render recusou antes de trocar a versão, a produção continuou no ar e inalterada.

3 · API reiniciando em loop no GitHub Codespaces

Sintoma: ao rodar o E1 no Codespace, o banco ficava *healthy*, mas a API ficava em *health: starting* e reiniciava a cada ~17 s. O `curl` devolvia *Connection reset by peer*.

Investigação: o `docker events` mostrou `exitCode=1` (erro da aplicação, não falta de memória, que seria 137). O log filtrado revelou `FlywaySqlException … Connect timed out`: a API não alcançava o banco. Um container de teste na mesma rede (`pg_isready -h db`) também não alcançou (*no response*), o que confirmou que o problema era a **rede entre containers do Docker do Codespace**, e não o projeto. Prova disso é que o mesmo `compose.yaml` funciona no CI (Testcontainers) e a mesma imagem funciona no Render.

Solução: um `compose.override.yaml` local, só para o Codespace e fora do Git, que coloca a API na rede do container do banco (`network_mode: "service:db"`, `DB_HOST: localhost`). Com isso a API ficou *healthy* (figura 1). O `compose.yaml` oficial não foi alterado, e o README documenta o contorno. **Lição:** o diagnóstico em camadas (código de saída → log → teste de rede isolado) evitou mudar o código por um problema que não era dele.

4 · Aprovação obrigatória travando o próprio PR

Sintoma: o PR mostrava *Review required*, e o autor não pode aprovar o próprio PR. **Solução:** manter obrigatórios o pull request e o check `testes` e ajustar a exigência de aprovação. **Lição:** o gate que realmente protege a produção é o teste. Com mais integrantes ativos no repositório, a aprovação de um colega pode voltar a ser exigida.

11 Checklist dos critérios de avaliação

O enunciado diz que **"item sem evidência não pontua"**. Por isso, cada item da avaliação aponta para onde a evidência está, no repositório e neste relatório.

E1 · Container automatizado (3,0)

Item	Onde está	Evidência
Dockerfile multi-stage (compila e testa no 1º, só o artefato no 2º)	`Dockerfile`	Seção 4 · Figura 7 (job imagem)
`compose.yaml` com API + banco, volume, healthcheck e depends_on	`compose.yaml`	Figuras 1 e 2
Configuração fora do código, sem segredos no repositório	`.env.example`, `.gitignore`, `application.yml`	Seção 4.3
Sobe do zero em poucos comandos; prints do `ps` e da API	README → Como rodar	Figura 1
Justificativas: imagem base, multi-stage, volume, rede	README → Decisões	Seção 8

E2 · TDD (3,0)

Item	Onde está	Evidência
Story map com backbone e Release 1; 3 histórias em tarefas	`docs/story-map.md`	Seção 3
Unitário por tarefa de regra	`src/test/java/.../dominio`	Seção 5 (72 casos)
Ao menos 1 teste de integração com o banco	`BolsaRepositoryIT`	Seção 5
Ao menos 1 aceitação por história (Gherkin)	`src/test/resources/features`	8 cenários, 3 features
Histórico red → green	`git log`	`e2-historico-red-green.txt`
Testes na esteira; vermelho barra o merge	`.github/workflows/ci-cd.yml`	Figura 3
Justificativas: ordem por risco, o que ficou sem teste, uso de IA	README → Testes e Uso de IA	Seções 5 e 9

E3 · Deploy automatizado (3,0)

Item	Onde está	Evidência
Pipeline build → testes → imagem → deploy	`.github/workflows/ci-cd.yml`	Figura 7
Fluxo de branch aplicado (branches, PRs, main protegida)	PRs #1 e #2; Settings → Branches	Figuras 3 e 4

Item	Onde está	Evidência
Deploy automático em serviço público, segredos no cofre	`render.yaml`, GitHub Secrets	Figuras 5, 8 e 12
Execução vermelha que barrou o deploy	Actions → execução #2	Figuras 3 e 6
Execução verde que publicou	Actions → execução #4	Figura 7
URL no ar	hemocentro-api.onrender.com	Figura 8
Como fazer rollback (e rollback testado)	README → Pipeline e deploy	Figuras 9, 10 e 11

Justificativas e IA (1,0)

Item	Onde está	Evidência
Tabela de decisões (o quê, alternativas, porquê)	README → Decisões	Seção 8 (17 decisões)
Uso de IA declarado, com prompts principais	README → Uso de IA	Seção 9

Referências

- HUMBLE, J.; FARLEY, D. *Continuous Delivery*. Addison-Wesley, 2010. Cap. 5, o deployment pipeline.
- FOWLER, M. *Continuous Integration*. martinfowler.com, 2006.
- BECK, K. *Test-Driven Development: By Example*. Addison-Wesley, 2003.
- NORTH, D. *Introducing BDD*. Better Software, 2006.
- DOCKER DOCS. *Compose file reference*; *Multi-stage builds*. docs.docker.com.
- GITHUB DOCS. *Understanding GitHub Actions*; *Building and testing Java with Maven*. docs.github.com/actions.
- BRASIL. Ministério da Saúde. *Portaria de Consolidação nº 5, de 28 de setembro de 2017*, Anexo IV (procedimentos hemoterápicos).